Università Degli Studi Di Salerno
Dipartimento di Ingegneria dell'Informazione ed Elettrica e Matematica (DIEM)
Corso di Laurea in Ingegneria Informatica

Ingegneria del Software
Documento di Design del software
SISTEMA DI GESTIONE PER BIBLIOTECA UNIVERSITARIA

Docenti: P. Foggia – N. Capuano

Gruppo di Progetto N. 26
Martina Petrini 0612710198 - m.petrini@studenti.unisa.it
Valeria Lanzara 0612708102- v.lanzara10@studenti.unisa.it
Andrea Murano 0612710133 - a.murano@studenti.unisa.it
Luca Migliaccio 0612709713 - l.migliaccio4@studenti.unisa.it

INDICE

1. ARCHITETTURA DEL SISTEMA
2. DESIGN DELLE CLASSI DI DOMINIO
2.1 CLASSE LIBRO
2.2 CLASSE UTENTE
2. DESIGN DELLE CLASSI DI DOMINIO
2.1 CLASSE LIBRO
2.2 CLASSE

1. ARCHITETTURA DEL SISTEMA

1.1 MODEL-VIEW-CONTROLLER (MVC)

Il sistema è basato sull'architettura MVC (Model-View-Controller).
Questa architettura divide l'applicazione in tre componenti interconnessi:

- Modello (Model): rappresenta il nucleo dell'applicazione, contiene i dati e fornisce metodi e funzionalità per interagire e manipolare i dati ed è indipendente dalla view e dal controller.
  Sono, dunque, le classi che contengono le informazioni pure (Libro, Utente e Prestito).
- View (Interfaccia): La view è responsabile di far visualizzare i dati all'utente attraverso le interfacce di cui ne gestisce i componenti.
  La view è completamente all'oscuro del modello sottostante e del suo stato. Comunica solo con il controller per inviare input utente e ricevere istruzioni su come presentare i dati.
  Dunque è l'interfaccia utente realizzata con JavaFx.
- Controller: funziona da intermediario tra il modello e la view: comunica con il modello per richiedere dati o innesca cambiamenti basati sulle azioni dell'utente e aggiorna la view per riflettere i cambiamenti che avvengono nel modello o dalle interazioni con l'utente (input).
  Implementa la logica di controllo.
  Un singolo modello può avere più viste per le diverse presentazioni dei dati.
  È la classe centrale GestioneBiblioteca che riceve i comandi dalla View, applica le regole del dominio (vincoli sui prestiti) e si interfaccia con il Model e la Persistenza.

Questo modello architetturale permette lo sviluppo di software ben strutturati, manutenibili e estensibili (capacità di permettere estensioni significative alle loro capacità senza apportare sostanziali modifiche all'architettura) essendo un modello orientato alla modularità.
Viene usato per lo sviluppo di applicazioni web (insieme al modello client-server), applicazioni Desktop e applicazioni mobile.

1.2 INTERFACCIA DEL CONTROLLER

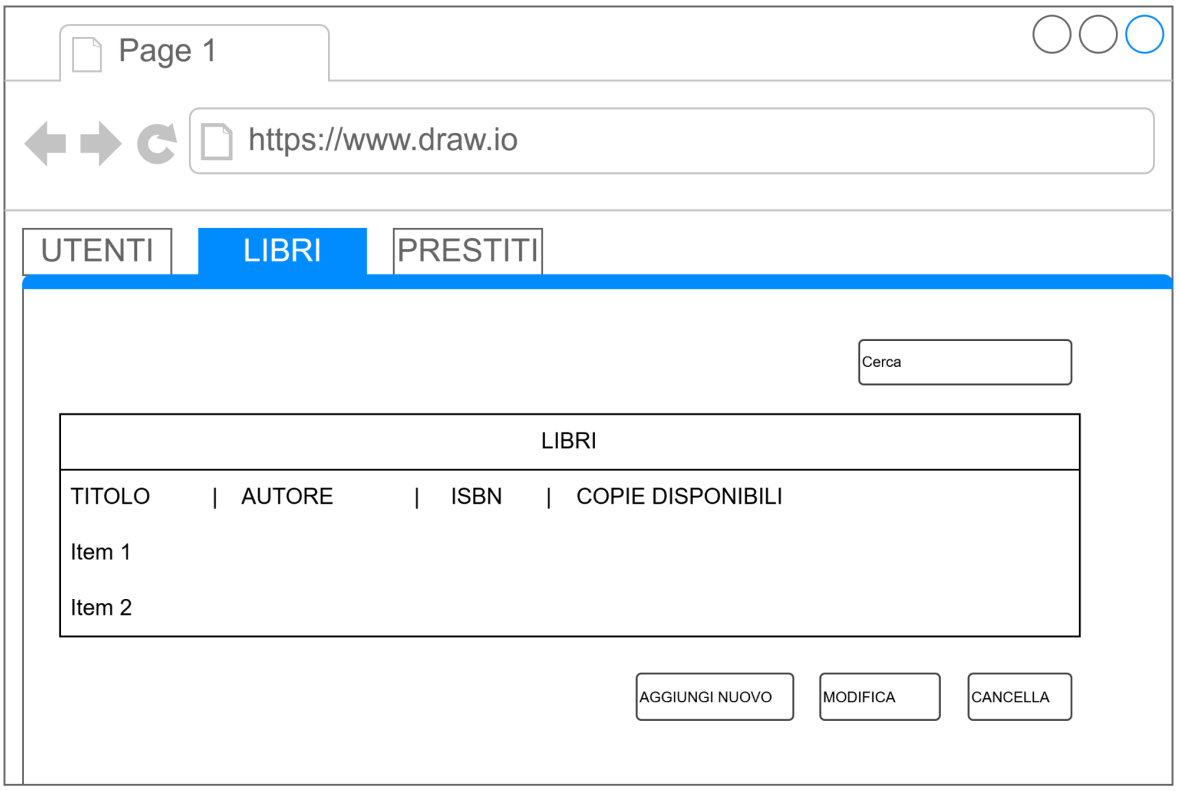
La classe GestioneBiblioteca è il punto di controllo centrale del sistema.
Agisce da intermediario tra l'interfaccia utente (View) e la Logica di Business (Modello e Persistenza).
La View è progettata per interagire con il sistema esclusivamente chiamando i metodi pubblici di questa classe.
Ciò, è fondamentale per l'architettura MVC, in quanto assicura che la presentazione dei dati (View) sia completamente separata dalla loro elaborazione (Controller).

La tabella seguente definisce nome, parametri e valore di ritorno di tutti i servizi offerti dal Controller per soddisfare i Requisiti Funzionali dell'applicazione.

Area Funzionale	(nome, parametri e valore di ritorno)	Descrizione
Gestione libri	boolean aggiungiLibro(Libro libro)	Inserisce un nuovo libro, verificando l'unicità dell'ISBN.
Gestione libri	boolean aggiornaLibro(String isbn, Libro datiAggiornati)	Aggiorna i dati di un libro esistente.
Gestione libri	List<Libro> cercaCatalogo(String query, String campo)	Restituisce la lista di libri cercati per titolo, autore o ISBN.
Gestione utenti	boolean aggiungiUtente(Utente utente)	Inserisce un nuovo utente, verificando l'unicità della matricola.
Gestione utenti	boolean aggiornaUtente(String matricola, Utente datiAggiornati)	Aggiorna i dati anagrafici di un utente esistente.
Gestione utenti	Utente cercaUtente(String matricola)	Cerca e restituisce un utente tramite matricola.
Gestione prestiti	List<Prestito> getPrestitiAttiviOrdinati()	Restituisce l'elenco dei prestiti attivi, ordinati per data, con la logica di evidenziazione dei ritardi.
Persistenza	void caricaArchivio()	Carica tutti i dati (Libri, Utenti, Prestiti) dal file persistente all'avvio.
Persistenza	void salvaArchivio()	Salva lo stato corrente del sistema sul file persistente.

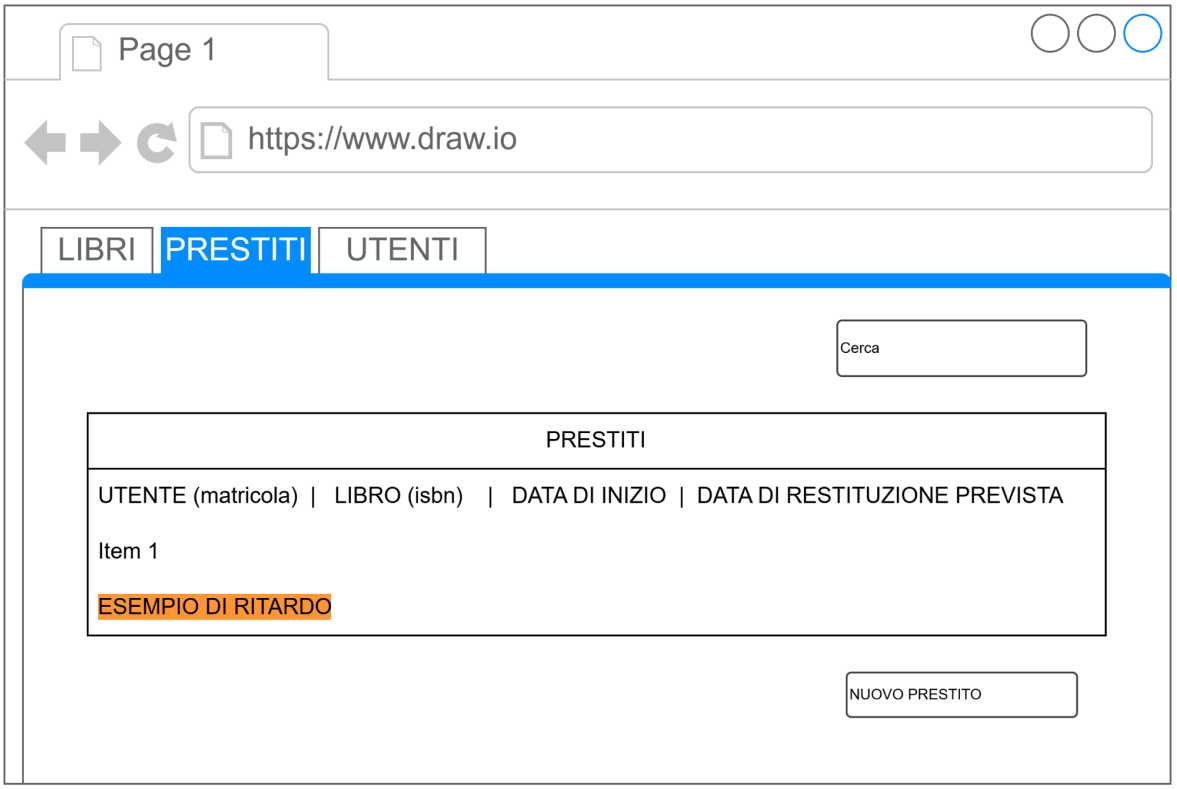
1.3 PROGETTAZIONE GUI E DIAGRAMMI DI FLUSSO

La View è realizzata con JavaFX e utilizza una navigazione a schede per separare le funzionalità di gestione.
(Si tratta di schizzi realizzati per rendere l'idea)

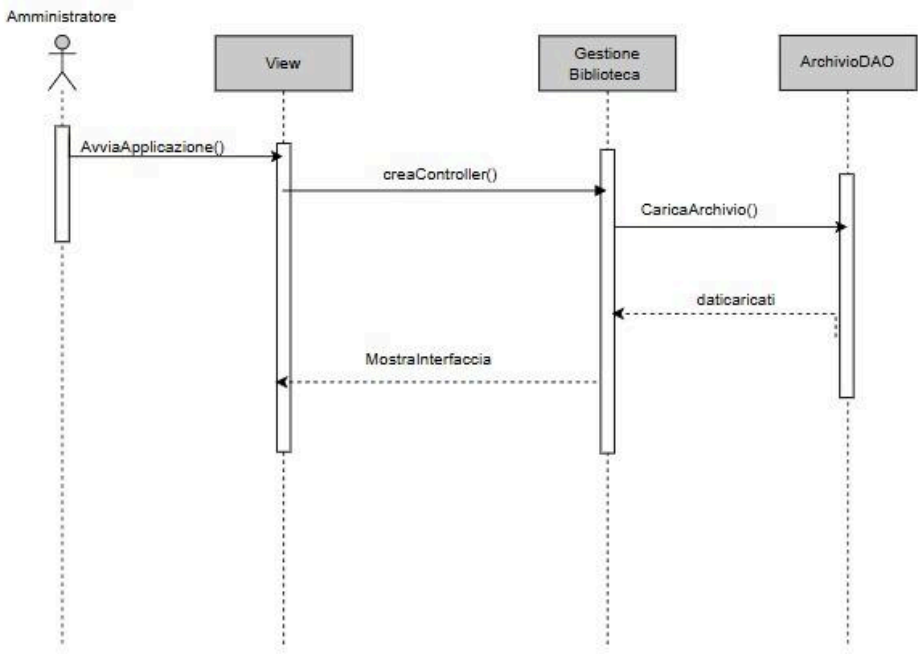


Questa interfaccia è dedicata alla gestione del catalogo libri.
Mostra una tabella con i campi chiave del modello libro (titolo, autore, isbn, copie disponibili). Sono presenti i pulsanti [AGGIUNGI NUOVO], [MODIFICA], e [CANCELLA] che, tramite il Controller GestioneBiblioteca, permettono l'esecuzione delle operazioni sul catalogo.
È inclusa una funzionalità di ricerca per titolo o autore.

La schermata gestione utenti presenta un layout analogo a quello dei libri, garantendo coerenza di interfaccia.
L'area centrale visualizza la tabella degli utenti con i campi anagrafici essenziali (matricola, nome, cognome, e-mail).
L'interfaccia include i pulsanti per la registrazione, modifica ed eliminazione degli utenti, delegando il controllo al controller GestioneBiblioteca che applica le verifiche di unicità della matricola.



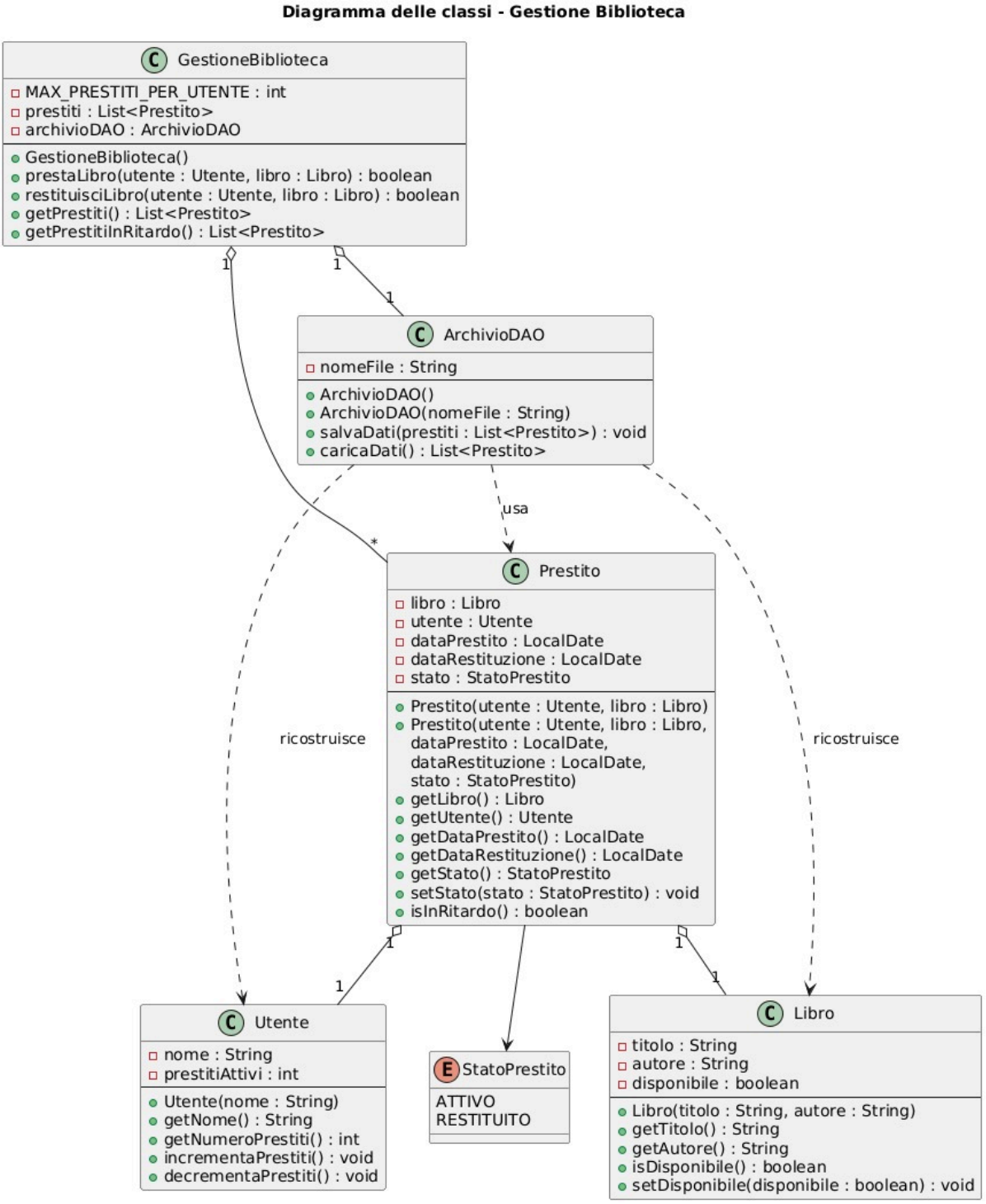
Questa è l'interfaccia per la logica centrale del sistema.
La tabella visualizza l'elenco dei prestiti attivi con i dettagli chiave (utente, libro, data prevista di restituzione).
Il sistema include un'area per la registrazione di un nuovo prestito ed una fondamentale nota di design: le righe in ritardo saranno evidenziate visivamente per soddisfare il requisito funzionale che richiede la logica di evidenziazione dei ritardi.



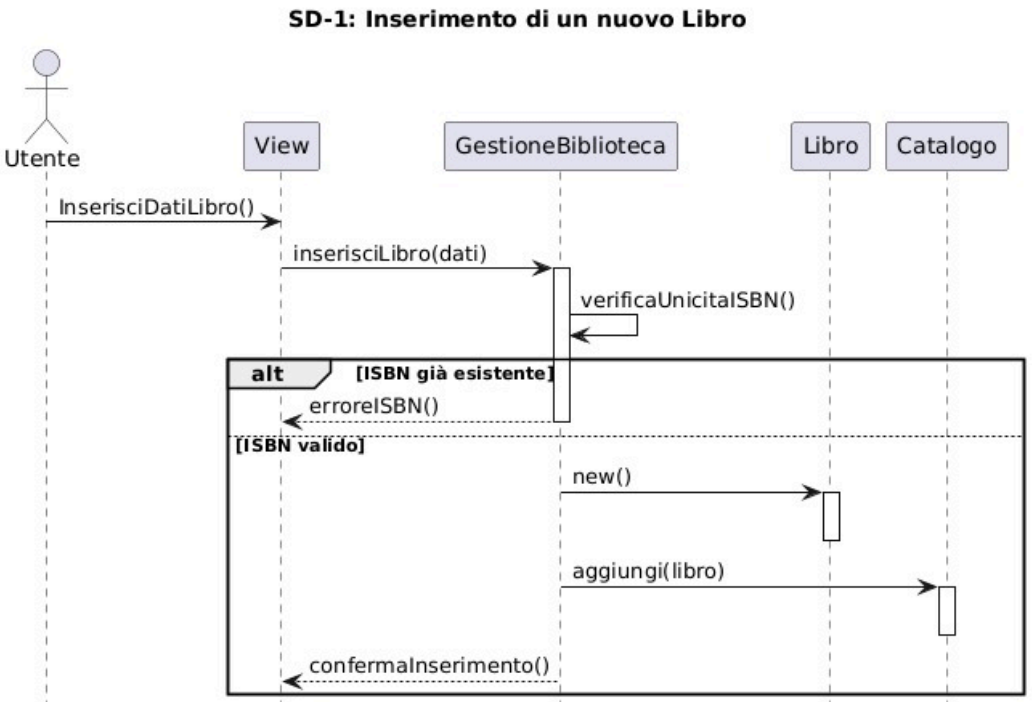
Questo diagramma illustra il flusso di controllo che si attiva all'avvio del sistema.
La MainApp (View) viene lanciata dall'Amministratore e inizializza l'oggetto GestioneBiblioteca (Controller).
Il Controller, a sua volta, invoca il metodo caricaArchivio() sulla classe di Persistenza (ArchivioDa0).
Una volta che tutti i dati (libri, utenti, prestiti) sono stati caricati, il controller istruisce la view a visualizzare l'interfaccia principale.

2. DESIGN DELLE CLASSI DI DOMINIO

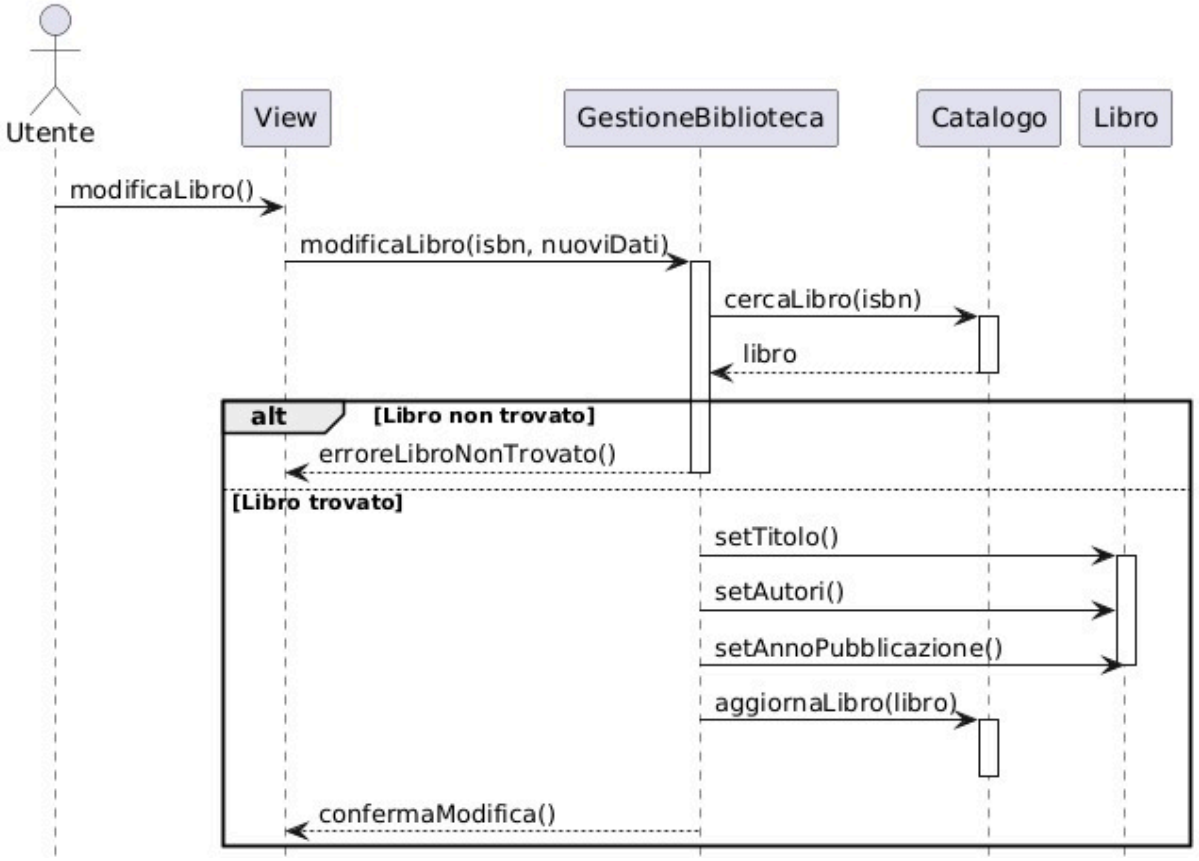
Questa sezione descrive la struttura interna delle classi del Modello, che contengono i dati e la logica di base per soddisfare i requisiti funzionali.



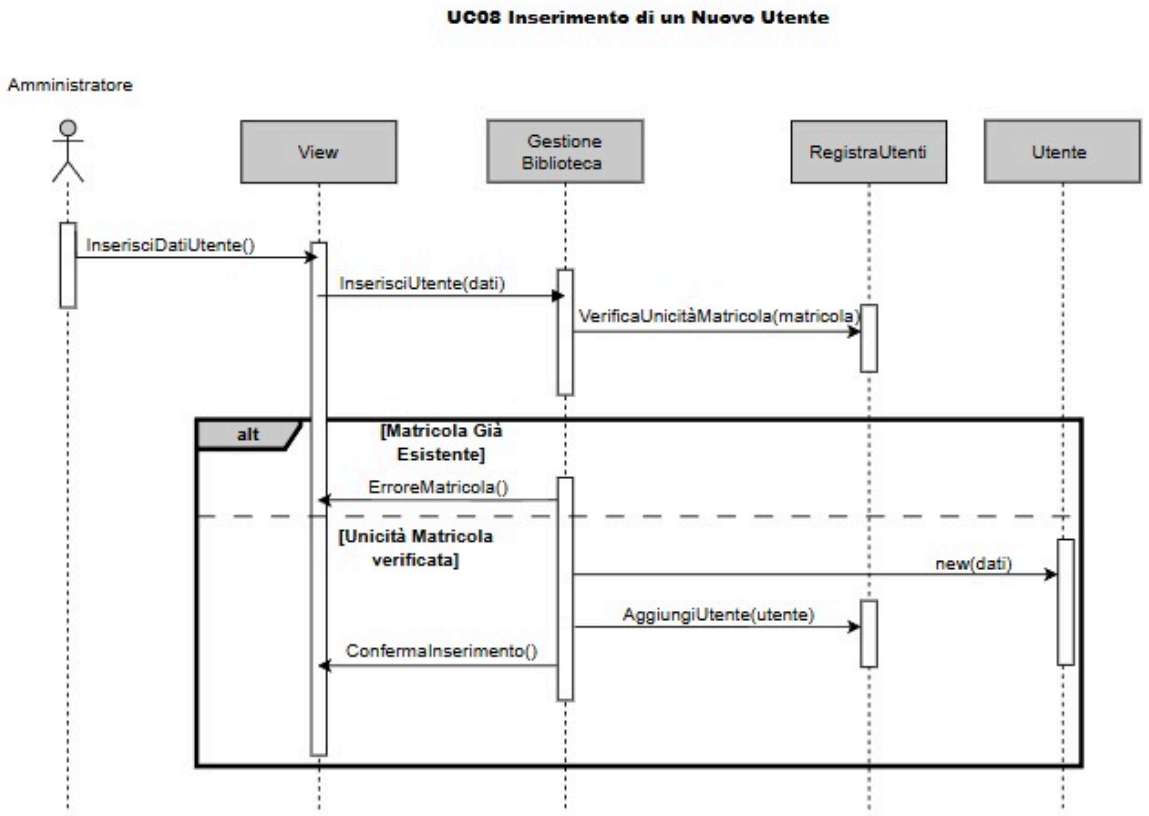
2.1 CLASSE LIBRO

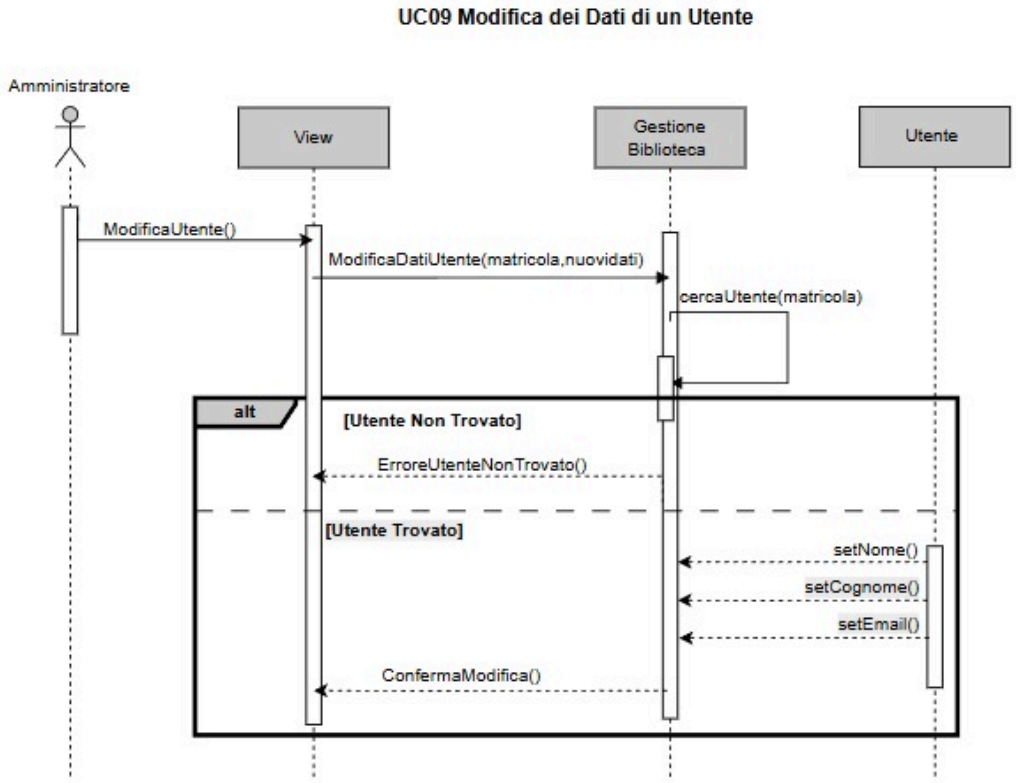
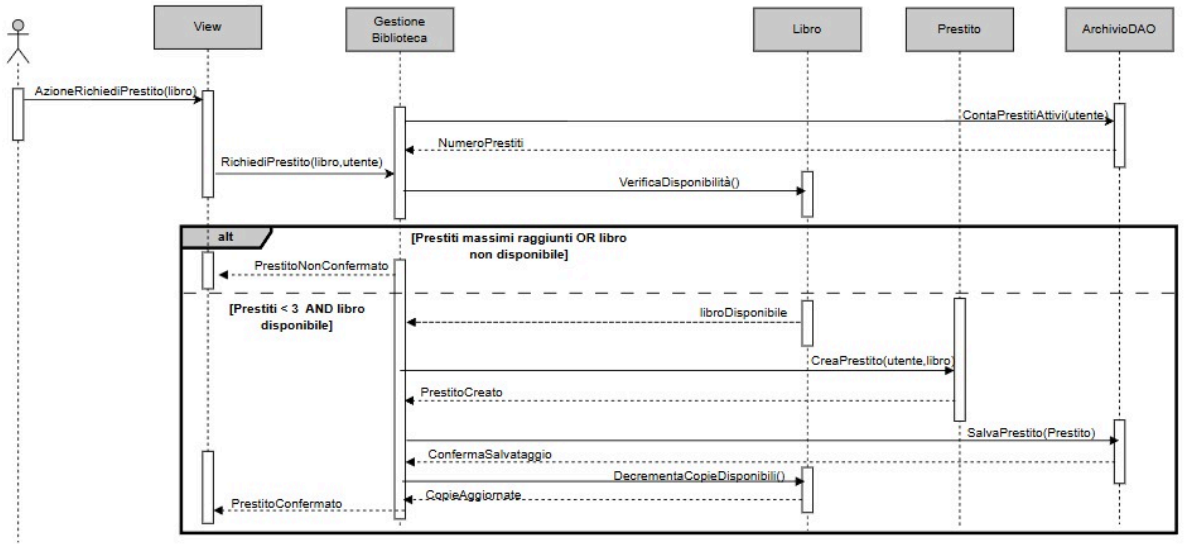


SD-2: Modifica dei dati di un Libro



2.2 CLASSE UTENTE





2.3 CLASSE PRESTITO